Evaluating Agricultural Loans and Expenditure Allocation in Nigeria Using Double Machine Learning

William Hankins[1], Alexis Villacis[2], Ani Katchova[3], Ivette Contreras[4]

[1]The Ohio State University. Email: hankins.181@osu.edu

[2]The Ohio State University. Email: villacis.9@osu.edu

[3]The Ohio State University. Email: katchova.1@osu.edu

[4] The World Bank. Email: icontreras@worldbank.org

Contents

Reproducibility Package README

The code within this replication package executes the analysis detailed in the paper "Agricultural Loans and Expenditure Allocation in Nigeria: Evidence from Double Machine Learning" using Stata, R, and Python. This package includes all code required to go from the raw survey microdata to the replication of all tables and figures in the analysis. The Stata master file (`00_master.do`) manages sequential execution of all data construction code. Python estimation is run in a second stage, these programs are computationally taxing and can take between minutes to over a day to run depending on specifications and hardware. Our primary objective is to ensure accessibility and credibility in replicating our research methodology and results.

Data Availability and Provenance Statements

The analysis uses two rounds of Nigeria's General Household Survey (GHS), a nationally representative panel survey conducted by the National Bureau of Statistics (NBS) in collaboration with the World Bank. The paper draws on Wave 4 (2018/19) and Wave 5 (2023/24). Raw GHS microdata are publicly available from the World Bank Microdata Library (`https://microdata.worldbank.org`); free registration is required.

In addition, the analysis uses pre-processed LSMS panel data compiled by the Evans School Policy Analysis and Research Group (EPAR), available at:

https://github.com/EvansSchoolPolicyAnalysisAndResearch/LSMS-Data-Dissemination/tree/main/Nigeria%20GHS

Statement about Rights

I certify that the author(s) of the manuscript have legitimate access to and permission to use the data used in this manuscript.

Summary of Availability

☑ All data are publicly available.

☐ Some data cannot be made publicly available.

☐ No data can be made publicly available.

Both the World Bank GHS microdata and the EPAR processed files are freely and publicly accessible at the links above. Raw data files (`.dta`). Place Wave 4 files in `Source Data/Wave 4/` and Wave 5 files in `Source Data/Wave 5/` before running any code.

Getting Started

Outline and Folder Structure

The reproducibility package consists of five top-level folders:

- `Source Data/` — GHS survey files (download from the links above).
- `Stata Code/` — All Stata do-files for data cleaning and table production. Final analysis dataset is written to `Stata Code/Stata Data Landing/DML Cleaned Data.dta`.
- `R Code/` — Single script (`FIES Index Calculation.r`) for Rasch-model food insecurity index; called automatically by the Stata master.
- `Python Code/` — DML estimation, sensitivity analysis, and figure scripts. Outputs are written to `Python Code/Python Table Landing/` and `Tables and Figures/`.
- `Tables and Figures/` — Final paper outputs (`.tex/.csv` tables, `.png` figures).

```
$ROOT/
|-- 00_master.do                  # Stata master (Stage 1 entry point)
|-- 00_figures.py                 # Python figure master (Stage 2)
|-- Source Data/Wave 4/           # GHS Wave 4 files
|-- Source Data/Wave 5/           # GHS Wave 5 files
|-- Stata Code/
|   |-- Stata Data Collection and Cleaning Scripts/
|   |    |-- Stata Control Covariate Collection and Cleaning Scripts/
|   |    |-- Stata Expenditure Calculations Scripts/
|   |    `--Stata Output Tables Scripts
|   `-- Stata Data Landing/DML Cleaned Data.dta  # Stage 1 output
|-- R Code/FIES Index Calculation.r
|-- Python Code/
|   |-- Python DML Scripts/       # fold_generator.py, learners.py,
|   |                             # helper_functions.py,
|   |                             # nuisance_function_residual_est.py,
|   |                             # DML Identification.py,
|   |                             # DML Benchmarking.py
|   |-- Pthon Figures Scripts/    # DML Figure A1, A2, A3, Benchmark, Summary
|   `-- Python Table Landing      # DML output location
`-- Tables and Figures/
```

How to Reproduce

PART 1: Stata — Data Construction. Open `00_master.do`, set the `global root` macro at line 1 to the repository root path, then run the file in full:

```
global root "C:/path/to/2nd-Year-Paper"
do 00_master.do
```

This installs required Stata packages, calls the R FIES script, and produces `Stata Code/Stata Data Landing/DML Cleaned Data.dta`. All subsequent stages read from that file only. Estimated runtime: ≤ 1 min.

PART 2: Python — Install Dependencies. Install all required packages with a single command (see Section 6.2 for the full `requirements.txt`):

```
pip install -r requirements.txt
```

PART 3: Python — Estimation and Output. Run the following scripts in order. No path changes are needed; each script resolves paths relative to its own file location.

- **PART 3.1 — Descriptive figures (A1, A2, A3).** `00_figures.py` (run from `$ROOT`)
  Produces kernel density figures. Saved to `Tables and Figures/`.
- **PART 3.2 — Summary statistics.** `Python Code/Pthon Figures Scripts/DML Summary Statistics.py`
  Produces farm-size quartile summary table; saved as `Tables and Figures/quartile_summary.csv`.
- **PART 3.3 — Main DML results.** `Python Code/Python DML Scripts/DML Identification.py`
  Estimates ATEs, HTEs (loan × farm size), and GATEs by farm-size quartile for each outcome. Output is a timestamped `.txt` file in `Python Table Landing/`. Set `N_REP=5` for a fast check; paper results use `N_REP=30`. Runtime: ≈20 min (`N_REP=5`); ≈2 hrs (2 Core Outcomes `N_REP=30`, 6 cores).
- **PART 3.4 — Sensitivity analysis.** `Python Code/Python DML Scripts/DML Benchmarking.py`
  Computes omitted-variable bounds by re-estimating with each control dropped in turn. Output CSV files written to `Python Table Landing/`. Runtime: ≈24 hrs (2 Core Outcomes, 21 benchmarks, `N_REP=30`, 6 cores).
- **PART 3.5 — Sensitivity forest plot.** `Python Code/Python Figures Scripts/DML Figure Benchmark.py`
  Reads benchmark CSVs from PART 3.3/4 and saves the forest plot.

Cache behavior. After PART 3.3/4 runs once, nuisance predictions are cached in `Python Table Landing/nuisance_cache/`. Subsequent runs load the cache automatically (`re_estimate_nuisance=False` by default), which is much faster. Set `re_estimate_nuisance=True` only if you change the dataset, learners, or fold/rep parameters.

Dataset List

Dataset	Description	Provided
`Source Data/Wave 4/Raw DTA Files/*.dta`	Raw GHS Wave 4 (2018/19) survey modules from the World Bank Microdata Library.	Yes
`Source Data/Wave 5/Raw DTA Files/*.dta`	Raw GHS Wave 5 (2023/24) survey modules.	Yes
`Source Data/Wave 4/final_data/*.dta`	EPAR LSMS processed data Wave 4 (linked above).	Yes
`Source Data/Wave 5/final_data/*.dta`	EPAR LSMS processed data Wave 5 (linked above).	Yes

List of Tables and Programs

Table / Figure	Program	Output file
Balance Table	`Stata Output Tables Scripts/ Balance Table.do`	`Tables and Figures/ balance_table.tex`
Main DML Results (ATE, HTE, GATE)	`Python DML Scripts/ DML Identification.py`	`Tables and Figures/ DML Identification_v*_*.txt`
Sensitivity Bounds	`Python DML Scripts/ DML Benchmarking.py`	`Tables and Figures/ benchmark_*.csv`
Summary Statistics	`Pthon Figures Scripts/ DML Summary Statistics.py`	`Tables and Figures/ quartile_summary_anova.csv`
Figure A1 (FIES KDE)	`Pthon Figures Scripts/ DML Figure A1.py`	`Tables and Figures/ Figure_A1.png`
Figure A2 (Farm-size KDE by treatment)	`Pthon Figures Scripts/ DML Figure A2.py`	`Tables and Figures/ Figure_A2.png`
Figure A3 (KDE by loan formality)	`Pthon Figures Scripts/ DML Figure A3.py`	`Tables and Figures/ Figure_A3.png`
Sensitivity Forest Plot	`Pthon Figures Scripts/ DML Figure Benchmark.py`	`Tables and Figures/ Figure_Benchmark.png`

Computational Requirements

Software Requirements

Stata

Stata/MP or Stata/SE version 17 or later. The following user-written packages are *automatically installed* by `00_master.do` on the first run:

Package	Source	Purpose
`univar`	SSC	Univariate statistics
`ietoolkit`	SSC	IE data management utilities
`winsor2`	SSC	Winsorizing continuous variables

R

R version 4.0 or later. Called via shell from `00_master.do` for the FIES index computation. R must be on the system `PATH` or its path must be set in the shell command in `00_master.do`. Required package: `RM.weights` (install via `install.packages("RM.weights")`).

Python

Python 3.9 or later (3.12 used for paper results). Table 3 lists all required packages with the exact versions used.

Table 3: Required Python packages

Package	Version	Purpose
`numpy`	1.26.4	Numerical arrays
`pandas`	2.2.2	Data manipulation; reads `.dta` via `read_stata`
`scikit-learn`	1.7.2	Ensemble learners and cross-validation
`doubleml`	0.11.1	DML Partial Linear Regression
`statsmodels`	0.14.6	Clustered OLS for GATE estimation
`matplotlib`	3.9.0	All figures
`scipy`	1.16.2	Kernel density estimation
`joblib`	1.5.2	Parallel nuisance estimation
`pyarrow`	22.0.0	Parquet I/O for nuisance cache

Installing Python Dependencies

Save the following as `requirements.txt` in the repository root, then run the installation command below.

`requirements.txt`:

```
numpy==1.26.4
pandas==2.2.2
scikit-learn==1.7.2
doubleml==0.11.1
statsmodels==0.14.6
matplotlib==3.9.0
scipy==1.16.2
joblib==1.5.2
pyarrow==22.0.0
```

Installation (run once from the repository root):

```
pip install -r requirements.txt
```

To install without pinned versions (use if the above causes conflicts):

```
pip install numpy pandas scikit-learn doubleml statsmodels matplotlib scipy joblib
    pyarrow
```

Key Configuration Parameters

Before running Stage 2, review these settings in `Python DML Scripts/DML Identification.py`:

Parameter	Paper value	Notes
`N_REP`	30	Set to $\leq$5 for a faster verification run
`k_fold_vector`	`[5]`	Cross-fitting folds
`N_WORKERS`	6	Set to available CPU cores
`re_estimate_nuisance`	`False`	`True` forces cache rebuild
`HTE`	`True`	Loan $\times$ farm-size interaction
`HTE_GATE`	`True`	Farm-size quartile (GATEs)

`N_REP` and `k_fold_vector` must be set to the same values in `DML Benchmarking.py`. The paper was run on Windows 11 with 6 physical cores and 16 GB RAM.

References

Lars Vilhuber, Connolly, M., Koren, M., Llull, J., & Morrow, P. (2022). "A template README for social science replication packages (v1.1)". *Zenodo*. https://doi.org/10.5281/zenodo.7293838

Evans School Policy Analysis and Research Group (EPAR). (2024). *LSMS Data Dissemination: Nigeria GHS*. https://github.com/EvansSchoolPolicyAnalysisAndResearch/LSMS-Data-Dissemination